Mastering JavaScript Interviews

A Comprehensive Guide to Core and Advanced JavaScript Concepts for Success

JavaScript Interview Question Bank

Here is a comprehensive interview-style question bank tailored directly to JavaScript topics. It transitions from core concepts to advanced mechanics, perfect for testing a full understanding.

Part 1: Core Fundamentals (Scope, Hoisting & Variables)

1. What is the fundamental difference between var, let, and const regarding scope and reassignment?

Answer:

- var is **function-scoped** (or globally scoped if declared outside a function) and can be redeclared and reassigned.
- let and const are **block-scoped** (restricted to the nearest pair of curly braces {}).
- let allows value reassignment, while const creates a read-only reference that cannot be reassigned.

2. Explain "Hoisting" in JavaScript. How does it affect variables declared with var versus let/const?

Answer:

- Hoisting is JavaScript's default behavior of moving declarations to the top of their current scope during the compilation phase before execution.
- Variables declared with var are hoisted and initialized with undefined.
- Variables declared with let and const are hoisted but **not initialized**.
- Function declarations are completely hoisted (both the name and the entire body).

3. What is the Temporal Dead Zone (TDZ)?

Answer:

- The TDZ is the period of time between a block's execution start and the point where a variable is declared using let or const. If you try to access the variable inside this zone, JavaScript will throw a ReferenceError.

4. What is Lexical Scope?

Answer:

- Lexical scope (or static scope) means that the scope of a variable is determined by its physical location within the source code at compile time. Nested functions have access to variables declared in their outer scope.

5. What is a closure? Can you give a practical example based on a Counter application?

Answer:

- A closure is a function that remembers and retains access to its lexical scope (outer variables) even when that function is executed outside its original scope.

```
function createCounter() {
  let count = 0; // Outer variable protected by closure
  return function() {
    count++;
    return count;
  };
}
const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

6. What is the difference between == and ===?

Answer:

- == is the **loose equality** operator. It performs **type coercion** (converts the operands to the same type before comparing).
- === is the **strict equality** operator. It compares both the **value** and the **type** without any conversion. 5 == '5' is true, but 5 === '5' is false.

7. Explain the difference between Primitive and Reference types in JavaScript.

Answer:

- **Primitive types** (String, Number, Boolean, undefined, null, Symbol, BigInt) store the actual value directly in the stack memory. They are compared by value.

- **Reference types** (Objects, Arrays, Functions) store a memory address pointing to the heap memory where the actual data resides. They are compared by reference pointer, not by structural value.

Part 2: Objects, Arrays & ES6+ Features

8. What is the difference between a shallow copy and a deep copy of an object? How do you achieve each?

Answer:

- A **shallow copy** duplicates the top-level properties, but nested objects still share the same memory reference. Achieved via the Spread operator ({...obj}) or Object.assign().
- A **deep copy** duplicates everything recursively, ensuring no nested references are shared. Achieved using JSON.parse(JSON.stringify(obj)) (with limitations) or the modern structuredClone(obj).

9. Given a nested user object, how can you extract a property using Object Destructuring with a fallback value?

Answer:

```
const user = { id: 1, profile: { name: 'Livesh' } };
const { profile: { name, role = 'Developer' } } = user;
```

10. How do map(), filter(), and reduce() differ from one another?

Answer:

- map() transforms each item in an array and returns a **new array** of the exact same length.
- filter() evaluates each item against a condition and returns a **new array** containing only items that match (true).
- reduce() executes a reducer function on each element to condense the entire array down into a **single output value** (like a sum, an object, or an accumulated string).

11. What do the array methods some() and every() return?

Answer:

- They both return a Boolean. some() returns true if **at least one** element passes the condition. every() returns true **only if all** elements pass the condition.

12. What are Symbols in JavaScript, and what is their primary use case?

Answer:

- A Symbol is a unique and immutable primitive data type. They are primarily used to create hidden, non-enumerable, completely unique keys for object properties that won't

conflict with any string keys (useful for internal architectural code or mixins).

Part 3: Asynchronous JavaScript & The Event Loop

13. JavaScript is single-threaded. How does it handle asynchronous tasks without freezing the UI?

Answer:

- JavaScript delegates time-consuming tasks (like setTimeout, network requests, or DOM events) to the environment's container (the browser's Web APIs or Node.js runtime). When these tasks complete, their callbacks are pushed to queues, and the execution engine picks them up when the Call Stack becomes empty.

14. Explain the roles of the Call Stack, Microtask Queue, and Macrotask (Callback) Queue in the Event Loop.

Answer:

- **Call Stack:** Executes synchronous JavaScript code sequentially.
- **Microtask Queue:** Holds callbacks from Promises (.then/catch/finally) and queueMicrotask. It has the highest priority.
- **Macrotask Queue:** Holds tasks like setTimeout, setInterval, and DOM events.
- **The Event Loop Rule:** The event loop continually checks the Call Stack. If empty, it flushes the *entire* Microtask Queue before pulling exactly *one* task from the Macrotask Queue.

15. What is Callback Hell, and how do Promises solve it?

Answer:

- Callback Hell refers to deeply nested, unreadable callback structures that look like a pyramid (坠). Promises flatten this structure by allowing asynchronous actions to be chained cleanly using .then(), passing errors downstream to a single .catch() block.

16. How does async/await improve Promise handling? Does it make code synchronous?

Answer:

- async/await is syntactic sugar built on top of Promises. It makes asynchronous code look and read like clean, sequential synchronous code. It does not block the main thread; underneath, it still relies on non-blocking Promise resolution.

17. How do you handle runtime errors when using async/await?

Answer:

```
async function fetchData() {
  try {
    let response = await fetch('https://api.example.com/data');
    let data = await response.json();
  } catch (error) {
    console.error("Failed to fetch data:", error);
  }
}
```

Part 4: Advanced Engine Mechanics & Execution Context

18. How is the this keyword determined in a regular function versus an arrow function?

Answer:

- In a **regular function**, this is dynamic and depends entirely on *how* the function is invoked (e.g., as a method of an object, standalone globally, or with call/apply/bind).
- In an **arrow function**, this is lexical. It does not possess its own this context; instead, it inherits it from its enclosing parent scope at declaration time.

19. What is a Higher-Order Function? Give an example.

Answer:

- A Higher-Order function is any function that accepts one or more functions as arguments, or returns a function as its output result. Examples include map(), filter(), or custom callbacks.

20. Contrast Debouncing and Throttling. When would you use each?

Answer:

- **Debouncing** delays a function execution until a certain period of silence has passed since the last invocation. (Best for search bar auto-completes).
- **Throttling** limits a function execution to occur at most once every specific time interval. (Best for window resizing or page scroll tracking).

21. Briefly explain the Prototype Chain in JavaScript.

Answer:

- Every JavaScript object possesses an internal private link pointing to another object known as its prototype. When you request a property or method from an object, JavaScript searches the object itself first. If missing, it travels up this prototype link sequentially until it encounters the property or reaches null.

22. What causes a Memory Leak in JavaScript, and how does Garbage Collection work?

Answer:

- **Garbage Collection** automatically deallocates memory using the **Mark-and-Sweep** algorithm, freeing data that has become entirely unreachable from the global root.
- **Memory leaks** occur when references to unwanted pieces of data are accidentally retained (e.g., forgotten global variables, unremoved event listeners, or uncleared intervals).

23. What is the functional difference between Map/Set and WeakMap/WeakSet?

Answer:

- Keys in a WeakMap/WeakSet must be **objects** (or registered symbols), and their references are held weakly. If an object key has no other references pointing to it anywhere else in the code, the garbage collector can cleanly wipe it out, preventing potential leaks.

24. What are Generators (function*) and Iterators in JavaScript?

Answer:

- An **Iterator** is an object implementing a .next() method that yields values sequentially.
- A **Generator** is a specialized function that can pause its execution mid-way using the yield keyword and resume exactly where it left off when invoked again.

25. Explain the role of JavaScript Proxies and the Reflect API.

Answer:

- A **Proxy** wraps a target object, allowing you to intercept and custom-configure fundamental operations (like property lookups, assignments, or enumeration). The **Reflect** API provides default target behaviors matching the proxy traps, making forwarding operations straightforward.

Part 5: DOM, Environment & Architecture

26. What is Event Delegation and why is it beneficial?

Answer:

- Event Delegation is a performance optimization pattern where instead of assigning individual event listeners to hundreds of child elements, you attach a single listener to a common parent container. It leverages **Event Bubbling** to capture actions occurring inside target child nodes.

27. Compare ES Modules (ESM) to CommonJS module systems.

Answer:

- **CommonJS** uses require() and module.exports. It loads modules synchronously and is native to Node.js.
- **ES Modules (ESM)** uses import and export. It evaluates statically at compile time, allowing features like **Tree-Shaking** (removing unused code), and is native to modern browsers.

28. What are Service Workers, and how do they differ from Web Workers?

Answer:

- **Web Workers** run scripts in background threads completely isolated from the main thread, ideal for shifting complex computational tasks away from the UI.
- **Service Workers** act as network proxies positioned between the browser, web app, and network interface. They intercept network requests, handle programmatic asset caching, and power offline application modes (PWAs).

29. Compare localStorage, sessionStorage, and Cookies.

Answer:

Storage Mechanism	Expiration	Max Size	Scope / Network
localStorage	Never expires unless cleared manually	~5-10MB	Client-only access
sessionStorage	Cleared when the tab or window closes	~5MB	Client-only access
Cookies	Set manually by server/client expiration date	~4KB	Sent to server automatically with HTTP requests

30. How do you inspect and resolve a runtime JavaScript exception or performance drop using Browser DevTools?

Answer:

- For exceptions: Open the **Console Panel** to read stack traces, or navigate to the **Sources Panel** to add breakpoints or conditional breakpoints directly on the offending code lines.

- For performance: Open the **Performance Panel**, run a recording during interaction execution, and inspect the Flame Graph to identify long-running, blocking tasks or memory layout shifts.